

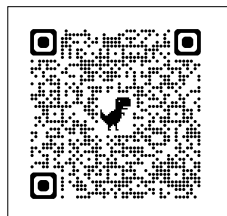
학생 과제 연구 안내

- 내용 : 과학 및 수학 · 공학 관련 주제를 선정하여 연구한다.
- 기간 : 3월 말~7월 중순
- 이수 : 계획서, 보고서, 소감문을 모두 제출 시
과학·수학 명장제 [과제연구] 중 학생과제연구를
수행한 것으로 인정함
- 대상 : 3학년(2~3인 1팀, 개인연구 불허)

- 참가 희망 학생은 아래 계획서를 작성하여
안내된 구글 폼에 파일을 업로드하기 바랍니다.
3월 18일(수) 16:50까지 제출, 시간엄수!

링크:

<https://docs.google.com/forms/d/e/1FAIpQLSdos2NAsm3Z7CnNq55vsdAN25mEj2-9qcwqHvEzqW1bgmWhOQ/viewform?usp=publish-editor>



※계획서를 평가하여 연구 진행 팀을 선발합니다.
최대한 정성껏 작성 바랍니다.

2026 < 학생 과제 연구 > 계획서

보고서 작성일 : 2026.3.10.

연번	학번	이름	지도교사
1	30716	윤은찬	(인)
2	30718	이상윤	
3	30802	강연우	

1. 연구 주제

연구 주제	연구 분야
Vulkan과 C# 기반 자체 3차원 그래픽 렌더러 구현 및 Unity 엔진과의 프레임 성능·최적화 기법 비교 연구	컴퓨터 그래픽스 / 컴퓨터 공학 / 실시간 렌더링 시스템

- 본인의 연구 주제에 대해 다음 항목을 기준으로 평가해봅시다.

연구 제목 : 제목만 봐도 연구의 성격과 목적을 파악할 수 있을 정도로 명료하고 구체적인가? ☒매우 그렇다 ☐그렇다 ☐그렇지 않다 ☐매우 그렇지 않다
주제의 적합성 : 고등학생 수준과 역량에 맞는 적절한 주제인가? ☒매우 그렇다 ☐그렇다 ☐그렇지 않다 ☐매우 그렇지 않다
연구 가치 : 연구할 가치가 충분히 있는 주제인가? ☒매우 그렇다 ☐그렇다 ☐그렇지 않다 ☐매우 그렇지 않다
현실성 : 연구 방법 및 절차가 주어진 시간과 장소에서 충분히 실현 가능한가? ☒매우 그렇다 ☐그렇다 ☐그렇지 않다 ☐매우 그렇지 않다

- 본인의 연구 최종 보고서를 읽은 독자는 무엇에 대해 새롭게 알 수 있을지 예상해서 작성하시오.

우선, Vulkan API를 사용해 3D 렌더링 파이프라인을 처음부터 직접 구현하는 과정과 그 구조를 이해할 수 있다. 평소 Unity나 Unreal Engine 같은 엔진을 통해서만 3D 그래픽을 접해왔다면, 엔진 내부에서 실제로 어떤 방식으로 그래픽 연산이 이루어지는지를 저수준 관점에서 들여다볼 수 있는 기회가 될 것이다.

또한 Frustum Culling, Occlusion Culling, Deferred Rendering 등 다양한 GPU 렌더링 최적화 기법들이 실제 성능에 어떤 영향을 주는지를 실험 데이터를 통해 직접 확인할 수 있다. 최적화 기법들의 효과를 동일한 실효 조건에서 정량적으로 비교한 사례는 흔하지 않기 때문에, 각 기법의 실질적인 성능 기여도와 기법들을 조합했을 때 나타나는 상호작용 효과를 파악하는 데 도움이 될 것이다.

마지막으로 직접 구현한 Vulkan 렌더러와 Unity 엔진의 성능을 같은 조건에서 비교한 결과를 통해, 상용 엔진의 추상화 레이어가 성능에 어떤 영향을 미치는지, 그리고 저수준 API를 직접 다루는 방식이 어느 상황에서 유리하고 어느 상황에서 한계를 보이는지를 구체적으로 이해할 수 있다. 게임 엔진 구조에 관심이 있거나 고성능 실시간 렌더링 시스템 개발을 목표로 하는 독자에게 실질적인 참고가 될 것이다.

2. 선행 연구 계획

- 선행 연구 분석에 쓰일 <참고 문헌>을 찾고 목록을 작성하시오.(제목-저자명-출판일)

제목	저자	출판일	비고
Vulkan all the way: Transitioning to a modern low-level graphics API in Academia	Unterguggenberger et al.	2023.04	
Light Performance Comparison between Forward, Deferred and Tile-based Forward Rendering	Vladislav Polyakov	2020.09	
Vertex Chunk-Based Object Culling Method for Real-Time Rendering in Metaverse	Eun-Seok Lee Byeong-Seok	2023.06.09	
Vulkan 1.4.346 Specification	Khronos Group	2026.03.13	

3. 준비물

- 연구에 필요한 실험 도구 및 재료를 선정하고 목록을 작성하시오.

	품명	규격	수량	예상 가격	비고
1	PD PPS 65W 충전기	65W 3포트	2개	83000	
2	NEXT-CC6204U2 데이터 케이블	c to c 60w	4개	12000	
3					
4					
5					
6					
7					

4. 관련 이론

- 연구와 관련된 이론

1. 렌더링 파이프라인

3D 그래픽 렌더링은 크게 **Application** 단계, **Geometry** 단계, **Rasterization** 단계로 구성된다. **Application** 단계에서는 씬의 오브젝트 데이터와 Transform 정보(위치·회전·스케일)을 나타내는 행렬 데이터가 준비되고, **Geometry** 단계에서는 Vertex Shader가 각 정점에 대해 Model-View-Projection 변환(오브젝트 → 월드 → 카메라 → 클립)을 수행하여 화면 좌표계로 변환한다. **Rasterization** 단계에서는 변환된 삼각형 폴리곤을 픽셀 단위로 분해하고, 프래그먼트 셰이더(Fragment Shader)가 각 픽셀의 최종 색상을 계산한다.

변의 길이가 각각 1, 2, 3인 직육면체.

- 세팅

타겟 점점: $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix}$ (행렬 계산용)

Model: $x: +2 \mid y: +1 \mid z: -5$
(오브젝트 move)

$$\begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

View: $x: 0 \mid y: -1 \mid z: 2$
(카메라 move)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Projection: 근점 → 원점 → 4D압축
(원근법)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

• 클립 위치 (Clip) = $P \times V \times M \times$ 점점

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix}$$

∴ Clip 좌표 = $\begin{bmatrix} 3 \\ -4 \\ -4 \\ 1 \end{bmatrix}$

• Perspective Divide

W값으로 X, Y 값 나누기 → 최종 화면값

$x: \frac{3}{1} = 0.75$

$y: \frac{-4}{1} = -4$

2. 조명 모델

Blinn-Phong 모델은 표면의 색상을 Ambient, Diffuse, Specular의 세 가지 성분의 합으로 표현한다. **Ambient**는 씬 전체에 균일하게 적용되는 기본 밝기이며, **Diffuse**는 광원 방향과 표면 법선 벡터의 내적으로 계산되어 면의 밝기를 결정한다. **Specular**는 Half Vector(광원 방향 벡터와 시선 방향 벡터의 중간 방향 단위벡터)와 법선 벡터의 내적을 통해 광택 효과를 표현한다. 이 모델은 물리적으로 완전히 정확하지는 않지만 계산 비용이 낮아 실시간 렌더링에 적합하다.

PBR은 빛이 실제로 물체 표면에서 어떻게 반사되는지를 물리 법칙에 따라 계산한다. Metallic(금속인지 아닌지)과 Roughness(표면이 얼마나 거친지) 두 가지 파라미터로 재질을 표현하며, 보는 각도가 얇아질수록 반사율이 높아지는 Fresnel 효과도 자동으로 반영된다. Blinn-Phong보다 계산 비용은 높지만, 조명 환경이 바뀌어도 재질이 항상 현실적으로 보인다는 장점이 있다.

3. Shadow Mapping

Shadow Mapping은 광원 시점에서 씬을 렌더링하여 각 픽셀의 깊이값을 Depth Map에 저장한 후, 카메라 시점에서 렌더링할 때 각 픽셀이 Depth Map과 비교하여 그림자 여부를 판단하는 방식이다. 구현이 비교적 간단하고 실시간 렌더링에 적합하다는 장점이 있다. 다만 Depth Map의 해상도에 따라 그림자 경계에 계단 현상이 발생할 수 있으며, 광원 하나당 Depth Map을 별도로 생성해야 하기 때문에 광원이 많아질수록 비용이 커진다. 경계의 계단 현상은 PCF(Percentage Closer Filtering)를 통해 어느 정도 완화할 수 있다.

Ray Tracing은 카메라에서 출발한 광선이 물체에 부딪혔을 때 광원까지 다시 광선을 쏘아 그 경로가 막혀 있는지를 확인하는 방식으로 그림자를 계산한다. 빛의 경로를 물리적으로 추적하기 때문에 부드러운 그림자 경계, 간접광에 의한 그림자 등 Shadow Mapping으로는 표현하기 어려운 현상도 자연스럽게 재현된다. 그러나 픽셀마다 수많은 광선을 추적해야 하므로 연산 비용이 매우 크고, 실시간 구현을 위해서는 성능 좋은 하드웨어 가속이 필요하다.

4. Environment Mapping

Environment Mapping은 오브젝트 표면에 주변 환경의 반사 효과를 표현하는 기법이다. 대표적으로 Cube Map(씬의 주변 환경을 6방향에서 촬영한 텍스처를 큐브 형태로 구성한 것)을 사용한다. 렌더링 시 카메라 방향과 표면 법선 벡터를 이용해 반사 벡터를 계산하고, 해당 방향의 Cube Map 텍스처를 샘플링하여 반사 색상을 결정한다. 이 기법은 동적 반사를 완벽히 재현하지는 못하지만(사전에 캡처된 정적 환경만을 반영하므로), 계산 비용이 낮아 실시간 렌더링에서 광범위하게 활용된다.

5. GPU 렌더링 최적화 이론

Frustum Culling은 카메라 시야 범위 밖에 있는 오브젝트의 드로우 콜을 CPU 단계에서 차단하는 기법으로, 씬의 오브젝트 수가 많을수록 효과가 크다.

Occlusion Culling은 다른 오브젝트에 의해 완전히 가려진 오브젝트의 렌더링을 생략하며, GPU의 Occlusion Query(오브젝트가 실제로 화면에 보이는지 여부를 GPU가 판별하는 쿼리)를 활용하여 구현한다.

Level of Detail(LOD)은 카메라와의 거리에 따라 메시의 폴리곤 수를 단계적으로 낮추어 버텍스 셰이더의 연산 부하를 줄인다.

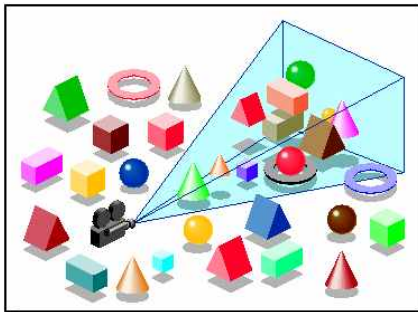
Instanced Rendering은 동일한 메시지를 반복 렌더링할 때 드로우 콜을 단일 호출로 통합하여 CPU-GPU 간 통신 비용을 절감한다.

Deferred Rendering은 렌더링 파이프라인을 Geometry Pass(씬의 위치·법선·색상 등 기하 정보를 G-Buffer에 저장하는 단계)와 Lighting Pass(저장된 G-Buffer를 기반으로 조명 연산을 수행하는 단계)로 분리하여, 실제로 화면에 보이는 픽셀에 대해서만 조명 계산을 수행함으로써 다수의 광원이 존재하는 환경에서 성능상 이점을 제공한다.

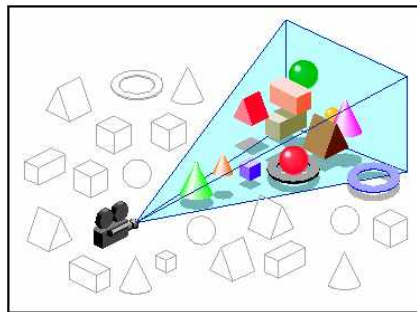
Mipmapping은 텍스처를 거리에 따라 미리 축소된 버전으로 샘플링하여 텍스처 앨리어싱을 줄이고 GPU 캐시 적중률을 높인다.

Early-Z Rejection은 깊이 테스트를 프래그먼트 셰이더 실행 이전에 수행하여 불필요한 셰이더 연산을 조기에 차단하며, Front-to-Back 렌더링 순서 정렬과 함께 적용할 때 효과가 극대화된다.

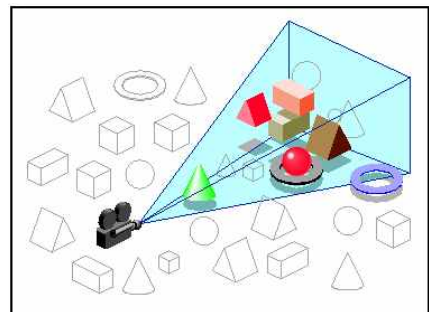
Uniform Buffer Object(UBO) 최적화는 셰이더에 전달하는 상수 데이터를 효율적으로 묶어 관리함으로써 CPU-GPU 간 데이터 전송 횟수를 최소화한다.



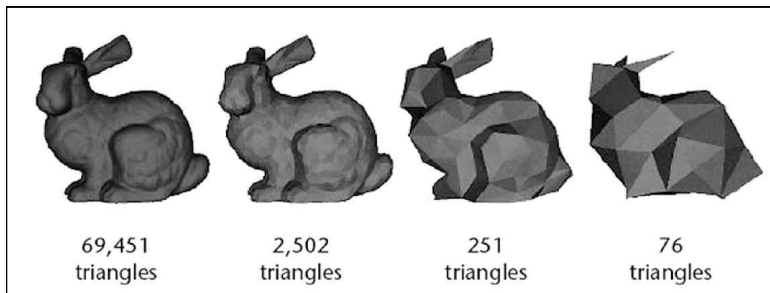
일반 렌더링
(모든 오브젝트 렌더링)



Frustum Culling 적용
(시야 밖 오브젝트 제거)



Occlusion Culling 추가
적용(가려진 오브젝트 제거)



LOD(Level of Detail)

단계별 폴리곤 수 감소

카메라 거리에 따라 메시의

폴리곤 수를 단계적으로 낮춰

버텍스 셰이더 연산 부하를 줄임.



Mipmapping 적용 전/후

텍스처 품질 비교

왼쪽: Mipmapping 미적용 시 원거리

텍스처에서 앨리어싱 발생

오른쪽: Mipmapping 적용 시 텍스처가

부드럽게 처리되며 GPU 효율 상승

6. 안티앨리어싱

3D 렌더링에서 오브젝트의 경계선이 계단처럼 끊겨 보이는 현상을 앨리어싱(Aliasing)이라고 한다. 이는 연속적인 3D 형태를 유한한 픽셀 격자에 표현하는 과정에서 필연적으로 발생하며, 이를 완화하는 기법을 안티앨리어싱이라고 한다. 대표적인 방식으로 SSAA, MSAA, FXAA, TAA가 있다.

SSAA(Super-Sample Anti-Aliasing) 는 화면 전체를 실제 해상도보다 더 높은 해상도로 렌더링한 뒤, 이를 축소하여 출력하는 방식이다. 여러 픽셀의 정보를 평균내기 때문에 네 가지 방식 중 화질이 가장 뛰어나다. 그러나 렌더링 해상도 자체가 커지므로 연산 비용과 메모리 사용량이 그에 비례하여 증가하며, 실시간 렌더링에서는 사실상 사용하기 어려운 수준의 부하가 발생한다.

MSAA(Multi-Sample Anti-Aliasing) 는 SSAA의 비용 문제를 개선한 방식이다. 화면 전체가 아닌 오브젝트의 경계선 픽셀에 한해서만 여러 샘플로 나누어 렌더링한 뒤 평균을 낸다. SSAA보다 연산 비용이 낮으면서도 경계선 품질은 유사하게 유지할 수 있다. 다만 샘플 수가 늘어날수록 메모리 사용량이 증가하며, Deferred Rendering 파이프라인과의 호환성 문제가 있어 별도의 처리가 필요하다.

FXAA(Fast Approximate Anti-Aliasing) 는 렌더링이 완료된 화면 이미지에 후처리로 적용하는 방식이다. 화면에서 색상 차이가 급격하게 변하는 부분을 경계선으로 감지하고 해당 영역을 흐리게 처리한다. 연산 비용이 매우 낮고 Deferred Rendering과도 자연스럽게 호환되어 구현이 간단하다는 장점이 있다. 그러나 경계선을 색상 차이로만 판단하기 때문에 경계선이 아닌 텍스처 내부의 세밀한 부분까지 흐릿하게 처리될 수 있다.

TAA(Temporal Anti-Aliasing) 는 현재 프레임뿐만 아니라 이전 프레임의 정보를 함께 활용하여 앨리어싱을 줄이는 방식이다. 프레임마다 샘플링 위치를 조금씩 다르게 설정하고 시간에 걸쳐 누적된 결과를 평균내기 때문에, 낮은 연산 비용으로도 높은 품질을 달성할 수 있다. 현재 대부분의 최신 게임 엔진에서 기본 안티앨리어싱 방식으로 채택하고 있다. 다만 오브젝트나 카메라가 빠르게 움직일 때 이전 프레임의 잔상이 남는 고스팅(Ghosting) 현상이 발생할 수 있으며, 이를 보정하기 위한 추가 처리가 필요하다.

	SSAA	MSAA	FXAA	TAA
처리 방식	고해상도 렌더링 후 축소	경계선 픽셀만 다중 샘플링	후처리 색상 감지	이전 프레임 누적 활용
화질	최상	높음	보통	높음
연산 비용	매우 높음	중간	매우 낮음	낮음
Deferred 호환	어려움	어려움	좋음	좋음
주요 단점	실시간 적용 어려움	메모리 사용량 증가	텍스처 뭉개짐	고스팅 현상

5. 연구 방법

- 연구 방법 및 절차를 구체적으로 작성하시오.

1단계. 연구 환경 구성

연구의 기반이 되는 3D 공간 구조를 설계한다. 기본적인 씬 구성 요소로서 월드 좌표계와 로컬 좌표계를 정의하고, 오브젝트의 Transform Matrix를 구현한다. 카메라 시스템은 View Matrix와 Projection Matrix를 수동으로 계산하여 구현하며, FOV, Near/Far Clipping Plane, Aspect Ratio 등의 파라미터를 조정 가능한 형태로 설계한다. 또한 C# 환경에서 Vulkan API를 호출하기 위해 Silk.NET 또는 Vulkan.NET 등의 바인딩 라이브러리를 선택하고 개발 환경을 통일하여 실험의 재현성을 확보한다.

2단계. Vulkan 기반 렌더러 구현 및 3D 렌더링 기술 적용

Vulkan의 핵심 렌더링 파이프라인을 직접 구성한다. 구체적으로는 Vulkan 앱 실행에 필요한 기본 환경 초기화, 화면에 이미지를 출력하기 위한 Swapchain 설정, 렌더링 단계 정의 및 결과물이 저장될 버퍼 구성, GPU에 보낼 렌더링 명령을 기록하고 실행하는 흐름을 구현한다. 이후 GPU에서 직접 실행되는 셰이더 프로그램을 작성하여 3D 메시를 화면에 렌더링하는 기본 파이프라인을 완성한다.

추가적으로 다음의 3D 렌더링 기술을 단계적으로 구현한다.

조명은 태양처럼 씬 전체에 일정한 방향으로 빛을 비추는 Directional Light와, 전구처럼 특정 위치에서 사방으로 빛을 방출하는 Point Light를 구현한다. 조명 계산은 단순히 그럴듯하게 보이도록 근사하는 방식 대신, 빛의 물리적 특성을 반영한 PBR(Physically Based Rendering) 방식을 적용한다. 이를 통해 재질이 얼마나 금속에 가까운지(Metallic)와 표면이 얼마나 거친지(Roughness)를 파라미터로 설정하여 금속, 유리, 플라스틱 등 다양한 재질을 현실적으로 표현한다. 또한 물체를 비스듬한 각도에서 볼수록 반사율이 높아지는 Fresnel 효과도 함께 적용한다. 주변 환경 전체를 광원으로 활용하는 IBL(Image-Based Lighting)은 일정에 여유가 있을 경우 추가로 구현할 예정이다.

그림자는 Shadow Mapping 기법으로 구현한다. 광원 시점에서 씬을 먼저 렌더링하여 각 픽셀까지의 거리를 저장해두고, 실제 카메라 시점에서 렌더링할 때 이 정보와 비교하여 그림자 영역을 판단한다. 그림자 경계가 계단처럼 끊겨 보이는 현상을 줄이기 위해 경계 주변 픽셀들의 결과를 평균내는 PCF 기법을 함께 적용한다. 빛의 경로를 물리적으로 추적하여 더욱 정확한 그림자를 표현하는 Ray Tracing 방식은 연산 비용과 하드웨어 요구사항을 고려하여 구현 가능 여부를 별도로 검토한다.

반사는 주변 환경을 6방향에서 촬영한 이미지를 큐브 형태로 구성한 Cube Map을 활용하여 구현한다. 카메라 방향과 표면의 법선 벡터를 이용해 반사 방향을 계산하고, 해당 방향의 Cube Map 이미지를 샘플링하여 반사 색상을 결정한다. 이를 씬 내 반사되는 금속 큐브·구 오브젝트에 적용한다. 일정에 여유가 있을 경우, Roughness 값에 따라 반사가 흐릿하게 퍼지는 효과까지 표현할 수 있도록 PBR 재질과의 완전한 연동을 목표로 확장한다.

반투명 유리 오브젝트는 보는 각도에 따라 반사율이 달라지는 Fresnel 효과와 빛이 물체를 통과하는 투과 처리를 함께 적용하여 구현한다. 안티앨리어싱은 오브젝트 경계에서 발생하는 계단 현상을 줄이기 위해 적용하며, MSAA(경계 픽셀을 여러 샘플로 나누어 평균내는 방식) 또는 FXAA(화면 전체에 후처리로 적용하는 방식) 중 성능과 품질을 비교하여 선택한다. 파티클 시스템(연기·불꽃 효과)은 일정에 여유가 있을 경우 추가로 구현할 예정이며, 각 파티클의 움직임·중력·소멸 등의 물리적 특성을 셰이더에서 직접 처리하는 방식을 검토한다.

3단계. 최적화 기법 모듈화 및 적용

각 최적화 기법을 독립적인 모듈로 구현하여 런타임 중 개별적으로 활성화·비활성화할 수 있는 토클 시스템을 설계한다.

각 모듈은 플래그 변수로 제어되며, 실험 시 특정 조합을 자유롭게 구성할 수 있도록 한다.

4단계. 실험 장면 제작

성능 측정의 공정성을 확보하기 위해 통제된 실험 씬을 제작한다. 씬은 크게 두 가지로 구성한다. 첫째는 격자 바닥과 단순 오브젝트로 구성된 디버깅용 씬으로, 조명·그림자·반사 등 각 기능의 동작을 명확하게 확인하기 위한 용도로 사용한다. 둘째는 지형과 원거리 오브젝트가 포함된 넓은 자연 월드 씬으로, Frustum Culling·LOD·Shadow Mapping 등 최적화 기법의 실제 효과를 측정하기 위한 용도로 사용한다.

씬 내 오브젝트는 큐브, 구멍 뚫린 큐브, 구, 다수의 반복 오브젝트, 반사되는 금속 큐브·구, 반투명 유리, 고폴리곤 오브젝트로 구성하며, 오브젝트 수·광원 수·그림자 적용 범위 등의 변인을 단계적으로 조절하여 저부하·중부하·고부하의 복수 테스트 케이스를 구성한다. Unity 엔진과의 비교를 위해 동일한 씬 구조를 Unity에서도 동등하게 재현하여 조건을 맞춘다.

5단계. 성능 측정 및 최적화 실험

각 실험 씬에서 최적화 모듈 조합을 다르게 설정하고, 동일 조건 하에 성능을 반복 측정하여 평균값을 구한다. 주요 측정 지표는 FPS, Frame Time, GPU·CPU 사용률, 드로우 콜 수, 메모리 사용량으로 설정한다. 측정 도구로는 Vulkan의 Timestamp Query, RenderDoc, NVIDIA Nsight 등을 활용한다. 아울러 동일한 조건에서 Unity 내장 렌더 파이프라인의 성능을 측정하여 자체 구현 렌더러와 비교한다.

6단계. 결과 정리 및 분석

수집된 성능 데이터를 표와 그래프로 시각화하고, 각 최적화 기법이 성능에 미치는 영향을 정량적으로 분석한다. 기법 간 상호작용 효과도 함께 검토한다. 자체 구현 렌더러와 Unity 엔진 간의 성능 차이를 비교·분석하고, 그 원인을 렌더링 파이프라인 구조의 차이 측면에서 고찰한다. 최종적으로 연구 결과를 바탕으로 경량 렌더러의 가능성과 한계, 그리고 향후 개선 방향을 보고서에 기술한다.

-가설-

Frustum Culling

씬 내 오브젝트 수가 많고 카메라 시야각이 좁을수록 FPS 향상 효과가 크게 나타날것임.

저부하 씬에서는 시야 밖 오브젝트 비율이 낮아 효과가 미미하지만, 오브젝트가 밀집된 고부하 씬에서는 컬링되는 드로우 콜 수가 급격히 증가하여 CPU 부하가 크게 감소할 것으로 예상된다.

Occlusion Culling

오브젝트가 서로 겹쳐 가려지는 비율이 높은 밀폐형 씬에서만 유의미한 성능 향상이 나타나고, 오픈 월드형 씬에서는 GPU Occlusion Query 비용이 이득보다 클 것임.

Occlusion Culling은 GPU 쿼리 레이턴시가 존재하므로, 가려지는 오브젝트가 적은 환경에서는 오히려 오버헤드가 발생할 수 있다.

LOD

원거리 오브젝트가 많은 씬에서 버텍스 셰이더 연산량이 감소하여 GPU 사용률이 낮아지지만, 근거리 오브젝트만 존재하는 씬에서는 효과가 거의 없을 것임.

LOD의 이점은 원거리 오브젝트의 폴리곤 수 감소에 있으므로, 씬 구성에 따라 효과 차이가 클 것으로 예상된다.

Instanced Rendering

동일 메시 반복 오브젝트가 100개를 초과하는 씬에서 드로우 콜 수가 1/N로 감소하며, GPU 사용률 변화보다 CPU 사용률 감소 효과가 두드러질 것임.

Instanced Rendering은 CPU-GPU 통신 비용을 줄이는 기법이므로, 병목이 CPU 쪽에 있을 때 효과가 집중된다.

Deferred Rendering

광원이 4개 이하인 씬에서는 G-Buffer 생성 비용으로 인해 Forward보다 오히려 느리고, 광원이 16개 이상인 씬에서는 Deferred가 확연히 우세할 것임.

Deferred의 핵심 이점은 광원 수에 비례한 연산량 증가를 억제하는 것이므로, 임계 광원 수 이상에서만 효과가 나타난다.

Mipmapping

고해상도 텍스처를 사용하는 씬일수록 GPU 캐시 적중률이 높아져 Frame Time 감소 효과가 클 것임.

Mipmapping은 텍스처 앨리어싱 감소뿐 아니라 캐시 효율 향상 효과가 있으므로, 텍스처 해상도가 높고 원거리 오브젝트가 많은 씬에서 효과가 극대화될 것이다.

Early-Z Rejection

오브젝트 겹침이 많은 씬에서 Front-to-Back 정렬과 함께 적용할 경우 프래그먼트 셰이더 실행 횟수가 줄어들지만, 오브젝트 수가 매우 많을 때는 정렬 오버헤드가 발생할 것임.

UBO 최적화

드로우 콜이 많은 씬에서 평균 FPS보다 Frame Time 편차(지터)가 줄어드는 효과가 두드러질 것임.

UBO 최적화는 평균 FPS보다 프레임 타임 안정성(편차)에 더 큰 영향을 줄 것으로 예상된다.

기법	주요 효과 예상 지표	효과가 큰 조건
Frustum Culling	CPU 드로우 콜 수 감소	오브젝트 많고 시야각 좁을 때
Occlusion Culling	GPU 렌더링 오브젝트 수 감소	밀폐된 씬, 오브젝트 밀집
LOD	버텍스 셰이더 연산량 감소	원거리 오브젝트 다수
Instanced Rendering	드로우 콜 수 1/N 감소	동일 메시 반복 100개 이상
Deferred Rendering	광원 처리 시간 감소	광원 16개 이상
Mipmapping	텍스처 샘플링 시간 단축	고해상도 텍스처, 원거리 씬
Early-Z Rejection	Fragment 연산 횟수 감소	오브젝트 겹침 많은 씬
UBO 최적화	Frame Time 편차 감소	드로우 콜 많은 씬